

Manual completo, passo a passo

BugIt

Agente de QA para Registro de Bugs no VS Code

Configuração inicial e uso diário — um guia detalhado, clique a clique, escrito para quem nunca fez nada parecido antes. Não é preciso conhecimento técnico. Siga na ordem e você estará registrando tickets de bug impecáveis ainda hoje.

Funciona no: VS Code — Windows (principal), macOS & Linux

O QUE VOCÊ VAI ENCONTRAR AQUI

Faça a **Parte 1** uma vez, na ordem. Tudo depois disso você pode consultar sempre que precisar. A **Solução de problemas** e o **FAQ** perto do final respondem às dúvidas mais comuns — confira-os antes de mandar um e-mail para o suporte.

Parte 1 · Configuração (faça isso uma vez)

- Passo 0 — Obtenha o GitHub Copilot
- Passo 1 — Instale o VS Code
- Passo 2 — Descompacte seu download do Buglt
- Passo 3 — Abra o Buglt no VS Code
- Passo 4 — Escolha o agente Buglt no chat
- Passo 5 — Ative sua licença
- Passo 6 — Aceite os termos (uma vez)

Passo 7 — Rode "Begin setup"

Passo 8 — Conecte seu tracker

Passo 9 — Salve seus logins

Passo 10 — Confirme que está tudo pronto

Parte 2 · Uso diário

Parte 3 · Deixe do seu jeito

Parte 4 · Atualizações, backups e segurança

Parte 5 · Solução de problemas e FAQ

Parte 6 · Licença e suporte

A única regra que mantém você seguro

O Buglt **nunca** registra nada sem antes te mostrar um preview e obter o seu "yes" digitado. Você está sempre no controle. Quer praticar com risco zero? Digite `dry run` e ele escreve o relatório inteiro **sem** registrá-lo.

Configuração inicial x uso diário

Você só segue toda a **Parte 1** uma vez. Depois disso, abrir o Buglt é rápido — e as atualizações são um único comando (`Update`). Veja a Parte 4.

PARTE 1 · CONFIGURAÇÃO — FAÇA ISSO UMA VEZ

Cerca de 15 minutos. Siga os passos na ordem — cada um depende do anterior. Não pule etapas.

0 Obtenha o GitHub Copilot

Você não consegue usar o Buglt sem isso

O Buglt é o "motorista" — o GitHub Copilot é o "motor" (a IA que realmente escreve os relatórios). Se você não tiver o Copilot funcionando no VS Code, nada do que vem a seguir vai ajudar. Configure isso primeiro.

O que é: o GitHub Copilot é um assistente de IA da GitHub (que pertence à Microsoft). Ele tem um nível gratuito para uso leve e um nível pago (cerca de \$10/mês) para uso intenso. Você vai instalá-lo no Passo 1 e fazer login.

1. Você vai precisar de uma **conta do GitHub**. Se não tiver uma, crie uma gratuita em github.com/signup.
2. O nível gratuito do Copilot já é suficiente para experimentar o Buglt. Se você registra bugs o dia todo, vale a pena o plano pago "Copilot Pro" — você pode fazer upgrade a qualquer momento em github.com/features/copilot.
3. Você vai realmente ativar o Copilot dentro do VS Code no Passo 1 — não precisa fazer mais nada aqui por enquanto.

1 Instale o VS Code

O VS Code é um aplicativo gratuito da Microsoft. É a "janela" onde o BugIt vive.

1. Abra seu navegador e acesse code.visualstudio.com.
2. Clique no grande botão azul **Download** (ele detecta seu sistema automaticamente).
3. Abra o arquivo baixado e instale-o — **aceite todas as opções padrão**, apenas clique em Next/Install até o fim.
4. Abra o VS Code assim que a instalação terminar.

Ative o GitHub Copilot dentro do VS Code

1. Na extrema esquerda do VS Code, clique no ícone de **Extensions** (parece quatro quadradinhos).
2. Na caixa de busca, digite **GitHub Copilot**.
3. Clique em **Install** em "GitHub Copilot" (e, se for oferecido, em "GitHub Copilot Chat").
4. Um aviso vai pedir para você **Sign in to GitHub** — clique nele e faça login com sua conta do GitHub na janela do navegador que abrir.

✅ **Você vai saber que funcionou quando:** um pequeno ícone do Copilot aparecer na parte de baixo do VS Code, e o painel de Chat abrir (o ícone de balão de fala à esquerda, ou pressione **Ctrl+Alt+I**).

2 Descompacte seu download do BugIt

Sua compra veio com um arquivo .zip (por exemplo, [bugit-qa-agent.zip](#)). Você precisa descompactá-lo — rodá-lo ainda zipado não funciona.

1. Encontre o arquivo .zip (geralmente na sua pasta **Downloads**).
2. **Clique com o botão direito** nele → escolha **Extract All...** (Windows) ou dê um duplo clique nele (Mac).
3. Escolha um local simples, como sua pasta **Documents**, e clique em Extract.
4. Agora você terá uma pasta **BugIt** com muitos arquivos dentro. Lembre-se de onde ela está.

Não coloque na OneDrive nem em uma pasta sincronizada

Pastas de sincronização na nuvem podem causar conflitos de arquivo. Sua pasta Documents ou a Área de Trabalho é perfeita. Também não renomeie os arquivos internos — deixe a pasta como está.

3 Abra o BugIt no VS Code

1. No VS Code, clique em **File** → **Open Folder** (menu superior esquerdo).
2. Navegue até onde você descompactou o BugIt (por exemplo, Documents).
3. Clique uma vez na pasta **BugIt** para selecioná-la, depois clique em **Select Folder**.
4. Se o VS Code perguntar "Do you trust the authors of the files in this folder?", clique em **Yes, I trust the authors**.

✅ **Você vai saber que funcionou quando:** você vir os arquivos do BugIt (pastas como [tools](#), [.github](#), e arquivos como [README.md](#)) listados na lateral esquerda do VS Code.

4 Escolha o agente Buglt no chat

1. Abra o painel de **Chat** do Copilot — clique no ícone de balão de fala à esquerda, ou pressione **Ctrl+Alt+I** (Windows) / **Cmd+Alt+I** (Mac).
2. No topo da caixa de chat há um pequeno **menu suspenso de modo** (pode estar escrito "Ask" ou "Agent").
3. Clique nele e escolha **Buglt QA Agent** na lista.

Não está vendo "Buglt QA Agent" na lista?

Confira se você abriu a pasta do Buglt (Passo 3), e não outra pasta — o agente carrega de dentro dela. Feche e reabra a pasta se for preciso.

5 Ative sua licença

O e-mail da sua compra contém uma **chave de licença** (um código longo com traços e um ponto). Ela desbloqueia o Buglt no seu computador.

1. Na caixa de chat, digite **hi** e pressione Enter.
2. Em seguida, o Buglt vai pedir sua chave. Por padrão, ele abre um **prompt mascarado no terminal** para que a chave nunca apareça na tela nem no histórico do chat — um pequeno painel de terminal abre na parte de baixo do VS Code; clique nele, cole sua chave ali e pressione Enter (nada do que você digitar ficará visível — isso é esperado, é um prompt no estilo senha).
3. Prefere colar direto na caixa de chat? Isso também funciona — basta dizer isso, ou colar como sua primeiríssima mensagem.

✓ **Você vai saber que funcionou quando:** o Buglt responder "✓ Activated — licensed to [seu nome/e-mail]."

Mantenha sua chave privada

Sua chave está vinculada a você. Não compartilhe, publique ou revenda — uma chave compartilhada pode ser desativada. Uma chave cobre apenas o(s) seu(s) assento(s). Trocar de computador depois é tranquilo (veja o FAQ).

6 Aceite os termos (uma vez)

Na primeira vez, o Buglt mostra um resumo curto e pede que você aceite antes de fazer qualquer outra coisa.

1. Leia o resumo curto que ele mostra (você revisa cada ticket antes de ser registrado; a segurança do seu projeto é responsabilidade sua; as proteções são auxílios, não garantias — e, como elas são seguidas por qualquer modelo de IA que estiver respondendo por você no Copilot, um modelo mais leve/gratuito pode ocasionalmente pular um passo ou precisar que um comando seja repetido, algo que um modelo mais forte não faria).
2. Responda **I agree** e pressione Enter.

✓ **Você vai saber que funcionou quando:** o Buglt confirmar e seguir para a configuração. Isso só é perguntado uma vez — ele lembra.

7 Rode "Begin setup"

Agora o Buglt se configura sozinho, entrevistando você em linguagem simples. Você nunca edita um arquivo de configurações manualmente.

Ele vai fazer perguntas curtas e ativar somente o que você escolher. A mais importante é sobre o seu tracker:

O que ele pergunta...	O que responder
Qual tracker você usa? (escolha um — obrigatório)	Onde sua equipe mantém os bugs. Jira e Azure DevOps recebem configuração guiada com mapeamento de campos testado — severidade/prioridade são definidas automaticamente para combinar com seu relatório. O Jira usa o Atlassian Rojo MCP com login pelo navegador; o Azure DevOps usa o MCP remoto da Microsoft com escopo de organização (uma prévia pública), também com login pelo navegador. Sentry, GitHub, Linear e Notion usam um endpoint conhecido que o BugIt configura e verifica ao vivo na sua conta — trate-os como experimentais até que essas verificações passem. Todos os outros (GitLab, Shortcut, YouTrack, Bugzilla, ClickUp, Asana, Trello) se conectam por meio do seu próprio MCP server compatível, que o BugIt ajuda você a configurar. Escolha o que você já usa.
Ferramenta de crash? (opcional)	O Sentry usa um endpoint conhecido que o BugIt verifica ao vivo na sua conta (experimental); Crashlytics, BugSnag, Raygun, Rollbar, App Center e Datadog se conectam pelo seu próprio MCP server compatível — só se sua equipe usar um.
Gestão de testes? (opcional)	TestRail, Xray, Zephyr, Qase.
Postar bugs no chat da equipe? (opcional)	Slack, Teams, Discord ou e-mail.
Especificações / conhecimento? (opcional)	Confluence, Notion, SharePoint, Google Docs, ou uma pasta local de arquivos Markdown neste repositório.
Idiomas	Seu idioma principal (padrão inglês). Ele faz uma pergunta simples de sim/não se você também registra em um segundo idioma — a maioria das equipes usa só um.
Recursos avançados? (opcional)	Ferramentas extras como resumo de triagem, desfazer e exportação offline — veja a Parte 2. Tudo opcional.

Dica: comece a partir de um template

Se o seu trabalho se encaixa em uma categoria, digite `preset.py game` (também `web-saas`, `mobile`, `enterprise`) para pré-preencher categorias e configurações sensatas. Você sempre pode mudá-las depois.

A configuração parou ou foi interrompida? É só falar de novo

Se o `Begin setup` parar no meio do caminho — você fechou o VS Code, ou ele simplesmente pausou — digitar `Begin setup` de novo retoma exatamente de onde parou. Ele não vai fazer você responder de novo perguntas que já respondeu. Para mudar algo de propósito mais tarde (adicionar um tracker, trocar uma escolha), é só dizer — por exemplo, "adicionar um tracker" — e ele reabre aquela parte do seletor.

Seu tracker conversa com o Buglt por meio de uma pequena ponte (seu nome técnico é "MCP server"). O Buglt configura isso **para você** — você só responde perguntas e clica em um botão. Você nunca edita arquivos.

8a · Deixe o Buglt configurar a ponte

1. Durante a configuração, o Buglt oferece para conectar seu tracker. Para Jira e Confluence (Atlassian RoVo) ele conhece a conexão guiada; para Sentry, GitHub, Linear e Notion ele conhece o endpoint padrão e o verifica ao vivo na sua conta — **confirme** quando ele perguntar, e ele roda as verificações antes de você depender disso (trate-os como experimentais até que passem).
2. Para ferramentas auto-hospedadas ou fornecidas pela organização, ele te diz em uma frase onde encontrar o endereço do seu MCP server compatível e pede para você **colar no chat** — depois ele escreve isso nas configurações.

8b · Ligue a ponte

1. Na lista de arquivos à esquerda, abra o arquivo `.vscode/mcp.json` (clique nele uma vez).
2. Lá dentro, procure um texto pequeno em cinza que diz `> Start | More...` — ele fica bem acima do nome de cada servidor. É pequeno e fácil de não perceber.
3. Clique em **Start** em cada servidor que o Buglt te disse para iniciar (geralmente só o seu tracker).

8c · Faça login quando for pedido

Na primeira vez que uma ponte se conecta, ela precisa provar que é realmente você. Uma de duas coisas acontece:

- **Uma janela do navegador abre** → faça login no seu tracker normalmente (comum no GitHub e no Azure DevOps).
- **Uma pequena caixa aparece no topo do VS Code** → ela pede seu e-mail e/ou um token de acesso. Cole e pressione Enter.

Onde eu consigo um token de acesso?

Um token é como uma senha de uso único que seu tracker fornece. Crie um no site do seu tracker e depois cole quando a caixa pedir. Aqui estão as páginas das ferramentas mais populares — faça login com a conta que você normalmente usa:

Seu tracker	Onde criar um token
Jira / Confluence (Atlassian)	<code>id.atlassian.com/manage-profile/security/api-tokens</code>
GitHub	<code>github.com/settings/tokens</code> (ou apenas faça login pelo navegador)
GitLab	<code>gitlab.com/-/user_settings/personal_access_tokens</code>
Sentry	sentry.io → Settings → Account → API → Auth Tokens
Linear	linear.app → Settings → API → Personal API keys
Azure DevOps	Geralmente faz login pelo navegador — nenhum token necessário.

Copie seu token assim que ele for exibido

A maioria dos serviços mostra um token novo apenas uma vez. Copie-o imediatamente e guarde em um lugar seguro até o Buglt salvá-lo para você (Passo 9). Escolha o prazo de validade mais longo que seu tracker oferecer, para não precisar refazer isso tão cedo.

8d - Confira se está conectado

Depois de iniciar uma ponte, o texto cinza muda para "Running" ou mostra um ponto verde. Para ter certeza, digite no chat:

VOCÊ DIGITA

Check status

✓ **Você vai saber que funcionou quando:** o Buglt listar seu tracker como conectado / saudável.

A ponte desliga quando você fecha o VS Code

Toda vez que você reabrir o VS Code, clique em **Start** no seu servidor novamente em `.vscode/mcp.json`. Seus logins salvos são lembrados (Passo 9) — você só reinicia a ponte. Esse é um comportamento do VS Code, não um problema do Buglt.

9 Salve seus logins

Para você nunca mais precisar procurar seu token, deixe o Buglt guardá-lo com segurança no cofre de credenciais embutido do seu computador.

VOCÊ DIGITA

save my tokens

Por padrão, o Buglt abre mais uma vez um **prompt mascarado no terminal** para o seu e-mail/token, para que ele nunca passe pelo histórico do chat — cole ali quando abrir (se preferir, você pode pedir para ele simplesmente receber pelo chat). De qualquer forma, ele salva tudo no cofre de credenciais do seu sistema operacional (Windows Credential Manager / macOS Keychain / Linux Secret Service) — nunca em um arquivo comum, nunca enviado a lugar nenhum. Na próxima vez que uma ponte pedir, é só digitar:

VOCÊ DIGITA

show my tokens

...e o Buglt mostra seus dados salvos ali mesmo no seu editor (só para você, nunca enviado a lugar nenhum) para copiar e colar. Chega de vasculhar o site do seu tracker.

10 Confirme que está tudo pronto

Peça para o Buglt checar tudo mais uma vez antes de você registrar de verdade:

VOCÊ DIGITA

Check readiness

Ele lista qualquer coisa que ainda esteja faltando (geralmente só "inicie sua ponte" ou "faça login"). Quando estiver tudo verde, você verá "✓ Setup complete."

Pronto, a configuração acabou — para sempre

Você não vai repetir a Parte 1. De agora em diante, abrir o Buglt é: abrir a pasta → iniciar sua ponte em `.vscode/mcp.json` → digitar `hi`. Para mudar qualquer escolha depois, é só digitar `Begin setup` e dizer o que quer mudar.

Sem GitHub Copilot? Existe um assistente de terminal

Se você não pode usar o Copilot, abra o Terminal embutido (Terminal → New Terminal) e rode `python tools/setup_wizard.py`. Ele faz algumas perguntas e escreve suas configurações sem a IA. Você também pode rodar o Buglt de forma standalone com sua própria chave de IA — veja o FAQ.

PARTE 2 · USO DIÁRIO

Depois de configurado, você trabalha totalmente pelo chat. Fale naturalmente, ou use os exemplos exatos abaixo. Digite `help` a qualquer momento para ver a lista completa.

Escrever um relatório de bug

A função principal do Buglt. Descreva o bug em uma frase simples; ele escreve o ticket completo, preenche automaticamente a versão e a categoria, verifica duplicatas, mostra um preview e registra depois do seu `yes`.

VOCÊ DIGITA

Write a bug report: o app fecha sozinho toda vez que toco em Salvar no iPhone

Ele redige um título, passos para reproduzir, resultado esperado vs. real, severidade e plataforma — e sempre define o próprio campo de prioridade/severidade do tracker para combinar, não apenas um padrão genérico. Quer mudar algo antes de registrar? É só dizer — por exemplo, "deixa a severidade alta" ou "adiciona que começou na última build."

Bugs rápidos (registro ágil)

Para formatos comuns (crash, layout, lentidão, ausência, bloqueio...), o formulário rápido pula algumas perguntas e preenche a categoria e a prioridade para você — e ainda assim roda a verificação completa de duplicatas primeiro.

VOCÊ DIGITA (EXEMPLOS)

Quick bug: crash na inicialização depois de uma atualização

Quick bug: layout botão sobrepõe o texto na tela de configurações

Quick bug: blocker não consigo passar da etapa de pagamento

Quick bug: slow o painel demora 20 segundos para carregar

Relatórios de bug de crash

Se você conectou uma ferramenta de crash (como o Sentry), o Buglt faz tudo acima além de relacionar seu relatório ao crash e buscar a stack trace para você.

VOCÊ DIGITA

Write a crash bug report: o jogo trava ao abrir o mapa

Comentários de verificação e encerramento

Depois de testar uma correção, o Buglt escreve um comentário organizado para postar no ticket. Ele escreve (e, opcionalmente, posta) o comentário — mas não muda o status do ticket. Você ainda fecha/resolve/reabre o ticket você mesmo no seu tracker.

Você digita	O que você recebe
<code>Close #123</code>	Pergunta qual cenário (correção confirmada, fechado a pedido, funciona como projetado, reabrir...) e então escreve esse comentário.
<code>Write a verification comment for 1.2.0 on Windows</code>	Um comentário de "correção confirmada" com os detalhes da build preenchidos.
<code>Write a reopen comment for 1.2.0 on Windows</code>	Um comentário de "o problema ainda acontece" (depois você reabre o ticket você mesmo).

Tradução

Traduza relatórios ou termos entre os idiomas configurados, usando exatamente as palavras da sua equipe (do seu glossário — veja a Parte 3).

VOCÊ DIGITA (EXEMPLOS)

Translate to ja: o botão de salvar não faz nada na tela de perfil
Translate this bug report to English: [cole o texto]

Consulte informações (especificações e glossário)

Se você conectou especificações ou preencheu seu glossário, faça perguntas ao Buglt sobre o seu projeto.

VOCÊ DIGITA (EXEMPLOS)

O que é [seu termo]?
Me explique o fluxo de checkout
O que há de novo nas especificações?

Deteção de duplicatas

Isso acontece automaticamente antes de cada registro — o Buglt pesquisa no seu tracker por tickets correspondentes (mesmo os que estão escritos de um jeito um pouco diferente) e te avisa, para que você nunca registre o mesmo bug duas vezes.

Recursos avançados (se você os ativou)

Digite isto	O que faz
<code>Triage digest</code>	Standup instantâneo: bugs abertos por área/severidade, os mais antigos sinalizados.
<code>Export bug to file</code>	Salva o relatório como Markdown/CSV/JSON em vez de registrá-lo (ótimo antes de configurar seu tracker).
<code>Undo last filing</code>	Reverte o último ticket/comentário, onde seu tracker permitir.
(automático)	Marcadores de SLA em bugs urgentes · campos específicos por categoria.

A lista completa de comandos

VOCÊ DIGITA

help

...mostra todo comando disponível dentro do agente, a qualquer momento.

Pratique com segurança quando quiser

Digite `dry run` e o BugIt escreve o relatório inteiro **sem** registrar nada — perfeito para aprender ou testar.

PARTE 3 · DEIXE DO SEU JEITO — ADICIONE O CONHECIMENTO DO SEU PROJETO

Assim que sai da caixa, o BugIt é um QA agent de estado em branco. Veja como ensinar a ele o seu projeto depois da compra. Sem programar — na maior parte, é só conversar com ele no chat. Tudo o que você adiciona fica no seu computador.

1 · As palavras da sua equipe (o glossário)

Isso deixa as traduções e o palpite de categoria precisos para o seu projeto.

1. Na lista de arquivos, abra `.github/glossary/terms.template.md`.
2. Preencha a tabela com as palavras da sua equipe — nomes de recursos, nomes de telas, nomes de itens, abreviações e (se você usa dois idiomas) suas traduções.
3. Salve o arquivo (`Ctrl+S`). Pronto — o BugIt usa exatamente esses termos a partir de agora.

Atalho

Ative o "glossary auto-seed" durante o `Begin setup` e o BugIt sugere termos a partir dos seus tickets existentes — você só aprova cada um.

2 · O estilo dos seus bugs (estilo da casa)

Quer que os tickets combinem exatamente com o formato da sua equipe — estilo de título, rótulos de severidade, campos obrigatórios? Você não descreve — você **mostra**:

1. No chat, diga: "adapte ao estilo da minha equipe."
2. Cole **um screenshot** de um ticket existente do seu tracker.
3. O BugIt estuda (localmente, com qualquer dado pessoal removido antes) e salva seu formato, seguindo-o em todo ticket futuro.

3 · Suas categorias, plataformas e campos

Basta dizer ao BugIt em linguagem simples e ele atualiza suas configurações para você:

VOCÊ DIGITA (EXEMPLOS)

Minhas categorias são Gameplay, UI, Áudio e Rede
A gente testa no Windows e no Xbox

4 • Suas próprias ferramentas (conexões personalizadas)

Usa uma ferramenta que não está na lista integrada? Conecte-a como se fosse uma integrada:

VOCÊ DIGITA

```
Add a custom MCP server
```

Dê um nome curto para ela e o endereço dela (precisa ser `https://`); o BugIt se conecta e verifica a saúde dela para você.

5 • Simplesmente corrija ao longo do caminho

O jeito mais fácil de todos: quando o BugIt redigir um ticket, corrija qualquer coisa que você não gostar — um rótulo, a redação, a severidade. Com "learn from your edits" ativado (recomendado), ele lembra da sua preferência e a aplica da próxima vez. O conhecimento do seu projeto se acumula naturalmente — e nada disso jamais sai do seu computador.

Tudo o que você adiciona é seu e fica local

Seu glossário, estilo da casa, categorias e preferências aprendidas ficam no seu computador, recebem backup antes de cada atualização e nunca são enviados a ninguém.

6 • Compartilhando sua configuração com sua equipe (licença Equipe)

Depois de configurar seu glossário, estilo da casa e escolhas de tracker do jeito que você quer, dê ao resto da sua equipe exatamente a mesma configuração em um único comando, em vez de repetir os passos 1–3 em cada máquina:

VOCÊ RODA (UMA VEZ, DEPOIS QUE SUA CONFIGURAÇÃO ESTIVER DO JEITO QUE VOCÊ QUER)

```
python tools/share.py export
```

Isso escreve `config.share.zip` — suas escolhas de tracker/crash/comunicação, glossário e estilo da casa, tudo empacotado junto, sem senhas nem tokens dentro. Envie para sua equipe.

CADA PESSOA DA EQUIPE RODA

```
python tools/share.py import config.share.zip
```

Eles recebem exatamente a sua terminologia, estilo de bug e configuração de tracker imediatamente — sem redigitar categorias, sem colar de novo um screenshot para o estilo da casa. A licença e as configurações do dispositivo de cada um permanecem intactas.

Se a importação mudar para onde os dados são enviados

Se uma configuração compartilhada adicionar uma nova conexão personalizada ou endereço de notificação, o BugIt mostra exatamente o que mudou antes de qualquer outra coisa acontecer — nada é aplicado silenciosamente. Ele também tira um snapshot da configuração atual da pessoa primeiro, então `Restore my settings` desfaz a importação se não for o que ela esperava.

PARTE 4 · ATUALIZAÇÕES, BACKUPS E SEGURANÇA

Recebendo atualizações (um comando)

Quando uma versão mais nova estiver disponível, o Buglt menciona isso uma vez quando você diz `hi`. Atualizações gratuitas da v1.x estão incluídas enquanto sua licença estiver ativa. Para instalar:

VOCÊ DIGITA

Update

1. O Buglt tira um backup novo dos seus arquivos primeiro.
2. Ele baixa a nova versão e verifica se ela é genuína e não foi adulterada (ela é assinada criptograficamente — uma atualização falsa ou alterada é recusada).
3. Ele instala sem mexer nas suas configurações, glossário, estilo da casa, licença ou tokens.
4. Ele pede para você recarregar o VS Code (`Ctrl+Shift+P` → digite "Reload Window" → Enter). Inicie um chat novo e você já está na versão nova.

Nunca é preciso apagar pastas

Diferente da primeira instalação, as atualizações são no local. Você nunca apaga nem baixa nada de novo, e sua configuração é sempre preservada e salva em backup.

O que é seguro editar manualmente — e o que não é

Seguro, e mantido em cada atualização: `config.json`, `.github/instructions/house-style.instructions.md` (veja a Parte 3 — essa é a forma suportada de personalizar o comportamento), `.github/glossary/terms.template.md`, `.github/instructions/learned.instructions.md` e `.vscode/mcp.json`.

Não é seguro — não edite estes manualmente

O arquivo do agente (`.github/agents/bugit-qa-agent.agent.md`), qualquer outro arquivo em `.github/instructions/`, e tudo em `tools/` são o próprio produto, não suas configurações. Uma atualização os **sobrescreve silenciosamente**, e — diferente dos arquivos acima — **não existe backup para restaurá-los**. O Buglt verifica isso na inicialização e novamente pouco antes de instalar uma atualização, e avisa você se encontrar alguma mudança — mas a regra mais segura é simplesmente não editá-los.

Backups e restauração

Você digita	O que faz
<code>Back up my settings</code>	Salva um snapshot local do seu config, glossário, estilo da casa, licença e conexões.
<code>List backups</code>	Mostra cada snapshot salvo com sua data.
<code>Restore my settings</code>	Reverte para um snapshot (e tira um snapshot do seu estado atual primeiro, então a própria restauração pode ser desfeita).

Automático antes de cada atualização

O Buglt sempre faz backup antes de instalar uma nova versão, então uma atualização nunca pode perder sua configuração. Se algo parecer errado, `Restore my settings` resolve.

Segurança — o que é protegido

✅ Nada sem o seu "yes"

Todo ticket e comentário é mostrado em preview; qualquer coisa irreversível exige uma confirmação digitada.

🔧 Dry-run = zero gravações

Diga `dry run` e o Buglt só lê — nunca registra, comenta ou notifica.

🔑 Nenhum segredo em arquivos

Tokens ficam no cofre de credenciais do seu SO, nunca em um arquivo comum, nunca enviados para nós.

🔒 Roda na sua máquina

Ele conversa apenas com as ferramentas que você conecta e com o seu próprio modelo de IA.

Seus dados e privacidade

A única coisa que o Buglt envia para a Taskivator são dados de licença/atualização: sua chave de licença, um ID de dispositivo com hash de mão única (um código embaralhado que não consegue te identificar) e — só se você tiver definido um na configuração — o rótulo de assento que você escolheu (por exemplo, um nome ou e-mail, para diferenciar seus próprios dispositivos; nunca precisa ser real). Seus relatórios de bug, especificações, glossário, configurações e tokens nunca saem do seu computador. Sem telemetria, sem analytics, nunca. Os detalhes completos estão no arquivo `PRIVACY.md` na sua pasta do Buglt.

Riscos e avisos importantes — por favor, leia

A IA pode errar

O Buglt redige relatórios com um modelo de IA, que pode interpretar mal detalhes ou inventar algo. Sempre leia o preview antes de dizer "yes". Você está aprovando o que será registrado.

Ele registra no seu tracker de verdade

Depois que você aprova, um ticket de verdade é criado. Praticando? Use `dry run` para escrever o relatório sem registrá-lo.

A segurança do seu projeto é sua

Como você usa o Buglt, e a segurança dos seus projetos, dados e trackers, são de sua responsabilidade. As proteções reduzem o risco, mas não substituem sua revisão.

O que o Buglt não faz

O mapeamento de campos testado (severidade/prioridade definidas automaticamente) é Jira + Azure DevOps (Azure DevOps pela prévia pública do MCP remoto da Microsoft); o Confluence usa a mesma conexão guiada da Atlassian; Sentry, GitHub, Linear e Notion são experimentais e verificados ao vivo na sua conta; todo o restante se conecta pelo seu próprio MCP server compatível (que você configura, com a ajuda do Buglt); ele usa a sua IA (Copilot ou sua própria chave OpenAI/Anthropic), não um modelo embutido; a remoção de dados sensíveis é feita com melhor esforço; e ele roda somente no VS Code. Uma licença = um dispositivo (Solo) ou cinco (Equipe). Os resultados e a consistência também dependem de qual modelo de IA está respondendo dentro do Copilot — um modelo mais forte segue as etapas de segurança de forma mais confiável do que um modelo bem leve/gratuito.

PARTE 5 · SOLUÇÃO DE PROBLEMAS E FAQ

Quase todo obstáculo se resolve em segundos — e a solução mais rápida geralmente é simplesmente perguntar ao assistente.

🌟 Tente isso primeiro — peça para a IA corrigir

Sempre que algo der errado — durante a configuração ou no uso do dia a dia — a correção mais rápida quase sempre é **perguntar ao assistente direto no chat**. Descreva o que aconteceu em palavras simples (por exemplo, "a configuração parou no meio do caminho" ou "a ponte do meu tracker não conecta") e peça para ele corrigir. A IA consegue diagnosticar e resolver a maioria dos problemas na hora. Por favor, tente isso antes de nos mandar um e-mail — resolve a grande maioria dos problemas em segundos.

Aplicando uma correção da IA: o botão "Keep"

Se o assistente corrigir algo mudando um arquivo, o VS Code mostra uma pequena barra Keep / Undo. Se você pediu a correção e está satisfeito com ela, clique em **Keep** — a mudança é aplicada só na sua própria cópia, no seu computador. Clique em **Undo** para descartá-la. Nada que você mantém com Keep é compartilhado com mais ninguém.

O que você vê	O que fazer
"Activate to start"	Por padrão, o Buglt abre um prompt mascarado no terminal para sua chave; cole ali (ou apenas cole no chat, se preferir). Sem chave? Consiga uma na loja.
"No tracker enabled"	Digite <code>Begin setup</code> e escolha seu tracker.
Sua ponte mostra um X vermelho / não conecta	Abra <code>.vscode/mcp.json</code> e clique em Start acima do servidor. Ainda travado? Digite <code>fix servers</code> e siga os passos.
Ele continua pedindo para você fazer login	Digite <code>show my tokens</code> — exibido localmente, só para você copiar — e cole o valor na caixa. (Primeira vez? Veja o Passo 8c.)
Ele não registra um bug	Digite <code>Check readiness</code> — ele lista exatamente o que está faltando (geralmente a ponte não foi iniciada ou você não fez login).
Algo parece quebrado depois de uma atualização	Digite <code>Restore my settings</code> para reverter.
O agente parece "fora do script"	Digite <code>Read and follow all your given instructions</code> , ou comece um chat novo. Isso pode acontecer com mais frequência em um modelo de IA leve/gratuito — um modelo mais forte no seletor de modelos do Copilot é mais consistente.
As respostas parecem genéricas / não específicas do projeto	Dê mais contexto, ou mostre um ticket existente durante o <code>Begin setup</code> para ele aprender seu estilo. Rode o <code>Begin setup</code> de novo para refinar.
Você quer mudar uma escolha da configuração	Digite <code>Begin setup</code> e diga o que quer mudar (por exemplo, "adicionar um tracker" ou "começar de novo") — ele reabre só aquela parte. Se a configuração foi simplesmente interrompida no meio, um <code>Begin setup</code> sozinho retoma de onde parou em vez de recomeçar.

Verificações de saúde integradas

`Check status` (minhas ferramentas estão conectadas?) · `Check readiness` (o que falta antes de eu poder registrar?). Para uma verificação mais profunda, abra Terminal → New Terminal e rode `python tools/selftest.py`.

Perguntas frequentes

Preciso saber programar?

Não. Se você consegue instalar um aplicativo e digitar uma frase, você consegue usar o BugIt. Ele cuida das partes técnicas para você.

Ele vai registrar bugs sem eu saber?

Nunca. Todo ticket e comentário é mostrado em preview e precisa do seu "yes" digitado. Use `dry run` para praticar sem nenhuma gravação.

Eu preciso iniciar a ponte toda vez?

Sim — abra `.vscode/mcp.json` e clique em Start quando reabrir o VS Code. Seus logins salvos são lembrados; só a ponte precisa ser reiniciada.

Posso usar em dois computadores?

Um de cada vez no Solo (cinco no Team). Trocar de dispositivo é tranquilo — uma licença pode trocar de dispositivo uma vez a cada 24 horas. Digite `license status` para verificar.

E se o servidor de licenciamento estiver fora do ar?

Você continua trabalhando. O BugIt roda com uma verificação em cache por até 7 dias offline, então uma queda — ou apenas um fim de semana sem internet — nunca te tranca fora no meio do plantão.

Eu preciso do GitHub Copilot?

É o jeito mais fácil. Se você não tiver, rode `python tools/setup_wizard.py` no Terminal para configurar, e o BugIt pode rodar de forma standalone com sua própria chave OpenAI/Anthropic (`python standalone/run.py`).

Meus dados são privados?

Sim. A única coisa que o BugIt envia para a Taskivator são dados de licença/atualização — sua chave, um ID de dispositivo com hash e (só se você tiver definido um) o rótulo de assento escolhido — sem telemetria, nunca. Seus tickets em si vão apenas para o modelo de IA e o tracker que você conectar, nunca para nós.

Posso editar os arquivos do BugIt?

Você deve personalizar as suas partes — seu glossário, estilo da casa e configurações (Parte 3). Só não desative o licenciamento/ativação. Se você encontrar um bug de verdade na própria ferramenta, mande um e-mail para o suporte para que ele seja corrigido para todo mundo na próxima atualização.

Com quais trackers de bug ele funciona?

Jira e Azure DevOps recebem configuração guiada com mapeamento de campos testado (severidade/prioridade definidas automaticamente) — o Jira pelo Atlassian RoVo MCP, o Azure DevOps pela prévia pública do MCP remoto da Microsoft. Sentry, GitHub, Linear e Notion usam um endpoint conhecido que o BugIt configura e verifica ao vivo na sua conta (experimental até que essas verificações passem); GitLab, Shortcut, YouTrack, Bugzilla, ClickUp, Asana e Trello se conectam pelo seu próprio MCP server compatível — o BugIt escreve o ticket e o registra por meio do conector que você ativar. Escolha o seu durante a configuração e troque a qualquer momento com `Begin setup`.

Ele vai pegar bugs duplicados antes de eu registrar?

Sim. Antes de qualquer coisa ser escrita, ele pesquisa no seu tracker por relatórios parecidos — mesmo os escritos de um jeito um pouco diferente — e mostra as correspondências, para você evitar abrir o mesmo bug duas vezes. Você ainda decide se quer seguir em frente.

Ele consegue escrever tickets em mais de um idioma?

Sim. Adicione um segundo idioma durante a configuração e todo relatório é escrito em ambos, mantidos em blocos separados, usando seu glossário para os termos do produto ficarem exatos — ele nunca improvisa uma tradução.

Posso anexar um screenshot?

Basta colar a imagem no chat. O BugIt lê, remove qualquer coisa sensível que perceber, e anexa ao ticket assim que você aprovar.

Ele esconde senhas e tokens nos meus relatórios?

Quando a remoção de dados sensíveis está ativada, ele limpa e-mails, tokens e endereços IP de todo rascunho antes de registrar. Você sempre vê o texto completo no seu preview primeiro.

Registrei um ticket por engano — dá para desfazer?

O preview e o "yes" digitado evitam a maioria dos enganos antes de qualquer coisa ser escrita. Se você ativou o log de auditoria, digite `Undo last filing`. Caso contrário, feche ou edite o ticket no seu tracker normalmente.

Ele pode me ajudar a verificar ou fechar um bug?

Sim. Peça um comentário de verificação e ele redige uma nota clara de aprovado/reprovado (nos seus idiomas) no ticket existente — você aprova antes de ser postado.

Posso usar para mais de um projeto?

Cada pasta do BugIt é configurada para o tracker de um projeto. Para outro projeto, rode o `Begin setup` de novo para apontá-lo para outro lugar, ou mantenha uma cópia separada por projeto.

Como eu recebo atualizações?

Quando uma nova versão sai, o BugIt menciona isso quando você inicia um chat. Digite `update` e ele instala no local — seu glossário, estilo da casa e configurações são mantidos, e ele faz backup deles primeiro.

O que acontece quando minha licença expira?

Você não é cortado imediatamente. O BugIt continua funcionando durante um período de carência de 14 dias e mostra quantos dias restam (digite `License status` a qualquer momento). Renove dentro desses 14 dias e você nunca é interrompido; quando o período de carência acabar, ative uma chave renovada para continuar registrando. Isso é separado do passe offline de 7 dias mencionado acima, que cobre apenas uma queda do servidor de licenciamento.

Se eu renovar ou comprar outra licença, os dias se acumulam?

Não — licenças não se acumulam. Ativar uma chave nova **substitui** a atual; o prazo dela começa do zero no dia em que você a ativa, e quaisquer dias não usados da chave antiga não são transferidos. Então ative uma renovação perto do fim do seu prazo atual, não antes.

PARTE 6 · LICENÇA E SUPORTE

Termos de licença (resumo em linguagem simples)

Os termos completos e vinculantes estão no arquivo `LICENSE` na sua pasta do BugIt — esse documento é quem manda. Em resumo:

Tópico	Em palavras simples
Licenciado, não vendido	Você compra uma licença para usar o BugIt, não a propriedade dele. A Taskivator mantém todos os direitos.

Seus assentos	Solo = um dispositivo por vez; Team = cinco. Sua chave é pessoal — não compartilhe, revenda nem publique.
Revogação	Uma chave compartilhada, reembolsada, contestada (chargeback) ou usada indevidamente pode ser revogada.
Personalize, mas...	Edite seu config, glossário e templates livremente — mas não desative nem contorne o licenciamento/ativação.
Sua responsabilidade	Como você usa o BugIt e a segurança do seu projeto são de sua responsabilidade. Você revisa e aprova todo relatório antes de ser registrado. As proteções são auxílios, não garantias.
"Como está", responsabilidade	Fornecido "como está", sem garantia. No limite que a lei permitir, a responsabilidade é limitada ao valor que você pagou, excluindo danos indiretos/consequentes.
Prazo e legislação	Uma licença anual que começa no dia em que você ativa, regida pelas leis do Japão. Reembolsos são tratados pela loja onde você comprou.
Renovações não se acumulam	Ativar uma chave nova substitui a atual — o prazo dela começa do zero na ativação e os dias não usados não são transferidos, então renove perto do fim do seu prazo. Quando um prazo termina, um período de carência de 14 dias permite que você continue trabalhando enquanto renova, antes que os registros parem.

Dois documentos na sua pasta

`LICENSE` (termos completos) e `PRIVACY.md` (declaração de privacidade). Ao ativar e usar o BugIt, você aceita a `LICENSE`.

Como obter ajuda

Por favor, confira primeiro a Solução de problemas e o FAQ acima — eles cobrem quase tudo. Depois, nesta ordem:

1 • Peça para a IA corrigir ★

Seu melhor primeiro passo: descreva o problema no chat e peça para o assistente corrigir. Se ele mudar um arquivo e parecer certo, clique em Keep para aplicar (só no seu PC).

2 • Rode uma verificação de saúde

`Check status` · `Check readiness` · ou `python tools/selftest.py` no Terminal.

3 • Restaure

`Restore my settings` desfaz uma mudança ou atualização ruim.

4 • Fale com um humano

Só se o guia e a IA não conseguirem ajudar: visite bugit.dev ou escreva para support@bugit.dev (o suporte é feito em inglês). Problema na configuração nos seus primeiros 7 dias? A gente te deixa funcionando ou ajuda com um reembolso pela loja.

Por favor, mantenha informações confidenciais do seu projeto fora dos e-mails de suporte

Cada projeto é diferente, e boa parte do seu trabalho pode ser confidencial. Se você nos escrever, descreva o problema do BugIt apenas em termos gerais — não cole código confidencial, detalhes de bugs, especificações, screenshots ou dados do seu projeto. A Taskivator não se responsabiliza por nenhuma informação confidencial enviada a nós, e qualquer coisa confidencial que recebermos é excluída imediatamente. Sempre que possível, peça ajuda ao assistente de IA primeiro — ele pode resolver a maioria dos problemas direto no seu editor.

Obrigado por escolher o BugIt.

Registre tickets impecáveis, pule as duplicatas e recupere seu tempo — um bug de cada vez.